Métodos Avançados de Programação MAP

– Padrões GoF – OBSERVER

Sabrina de F. Souto.

PROBLEMA

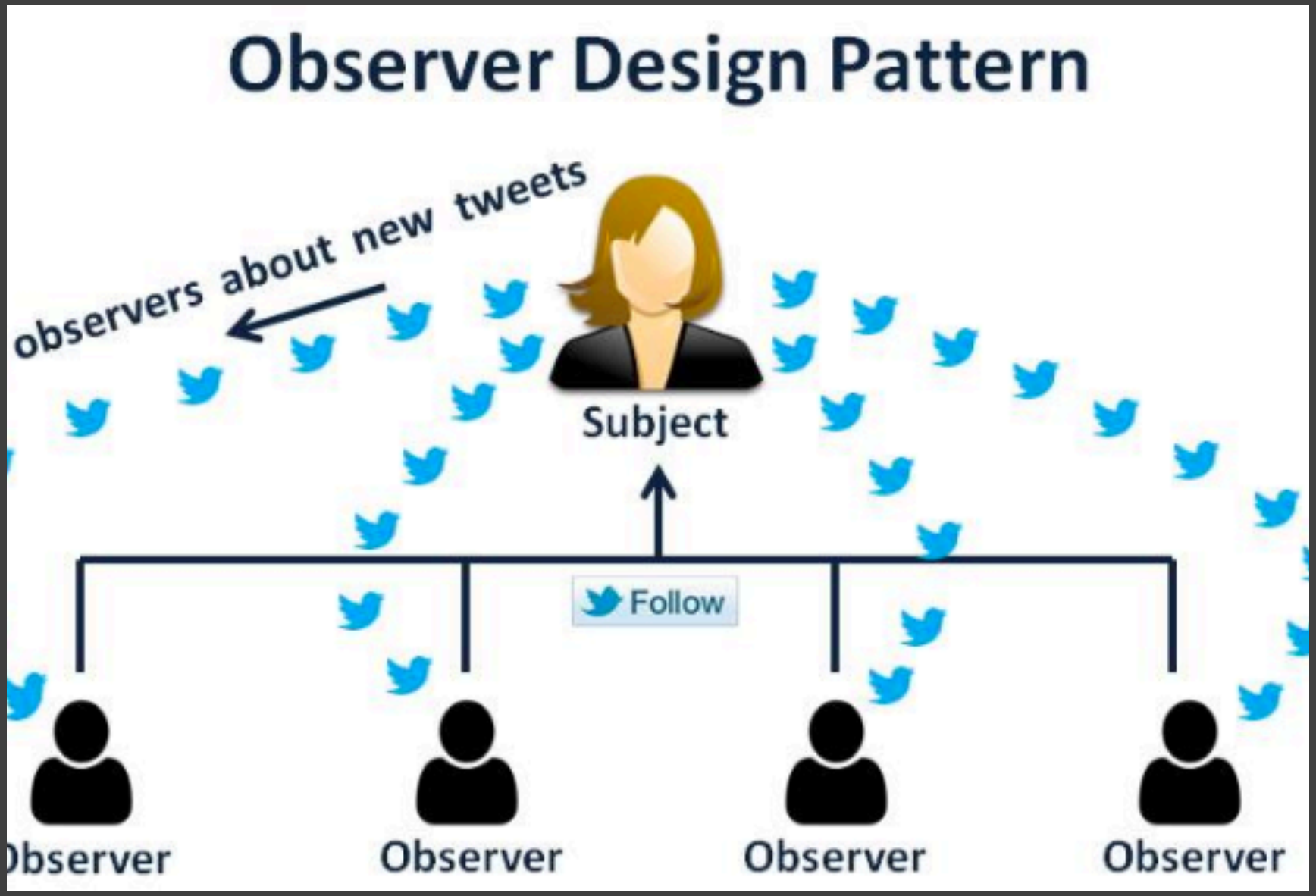
- Como acoplar objetos entre si:
  - De forma a que não se conheçam em tempo de compilação
    - Não queremos fazer "referênciaAUmObjetoConhecido.método()"
  - De forma a criar o acoplamento e desfazê-lo a qualquer momento em tempo de execução
- Implementação *muito flexível* de acoplamento de abstrações

Padrão Comportamental

OBSERVER

OBJETIVO

- O padrão Observer permite que objetos interessados sejam avisados da mudança de estado ou de eventos que ocorrem num outro objeto (*Publish* ← → *Subscribe*)
  - O objeto sendo observado é chamado de:
    - ***Source***
      - *Lança eventos*
  - O objeto que observa é chamado de
    - ***Listener***
      - *Recebe e processa eventos*


Observer Design Pattern
observers about new tweets
Subject
Follow
Observer
Observer
Observer
Observer

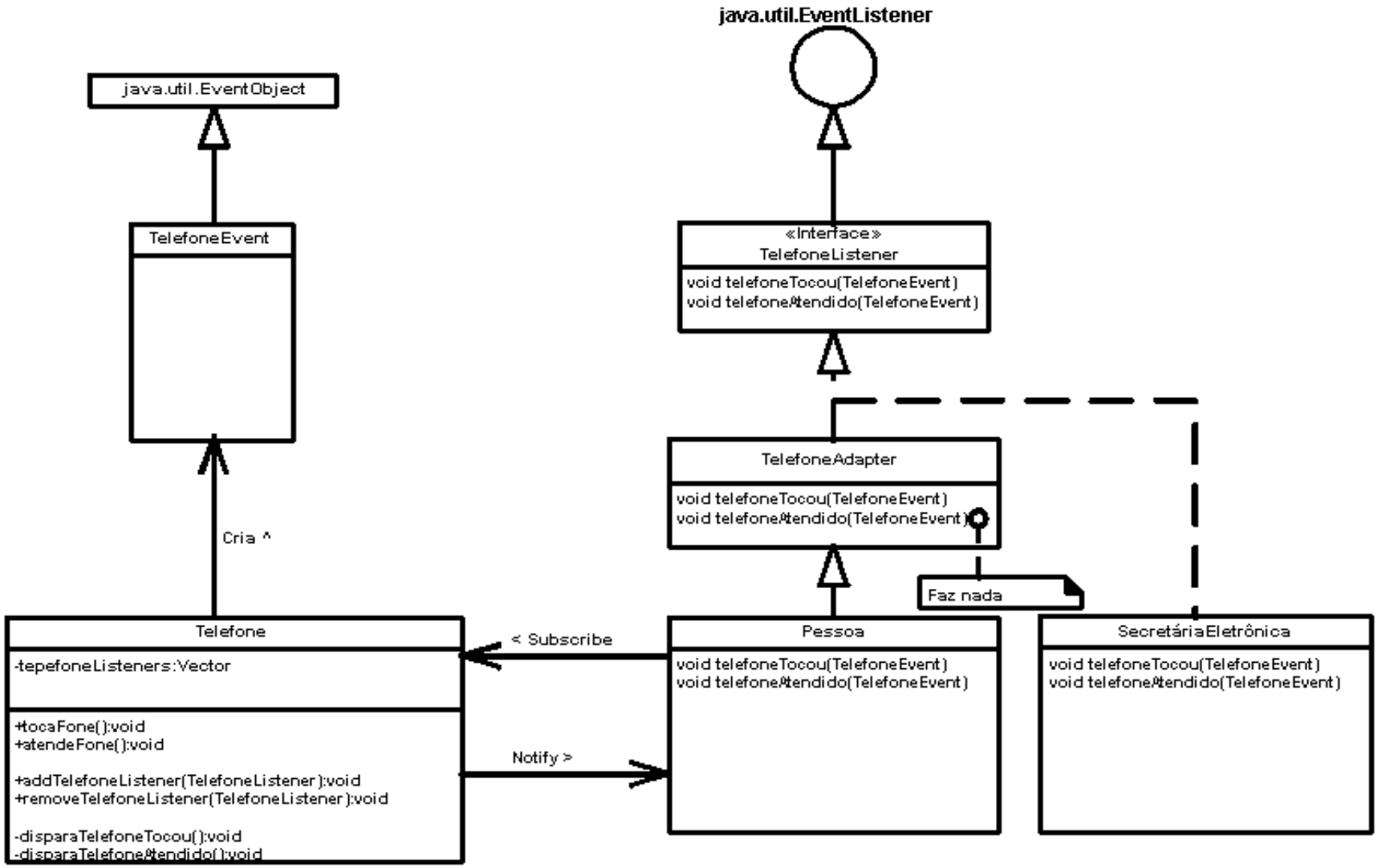
EXEMPLO

- Como projetar um sistema que modele um telefone e todos os objetos que poderiam estar interessados quando ele toca?
- Os objetos interessados poderiam ser:
  - Pessoas que estejam perto (na mesma sala), secretária eletrônica, FAX
- Os objetos interessados podem mudar dinamicamente

EXEMPLO

- Faça o telefone um Source
  - O objeto Source envia informação (eventos) para o objeto Listener pela chamada de um método do Listener
  - O Source deve ter uma referência ao Listener, para poder notificá-lo da mudança
- Detalhes:
  - Número/tipos dos Listeners não é conhecido em tempo de compilação
  - Os listeners poderão não fazer parte de uma mesma hierarquia de objetos
  - Não queremos criar um acoplamento forte entre Source e Listeners
- <u>Usando interfaces!</u>

ESTRUTURA


java.util.EventListener
java.util.EventObject
TelefoneEvent
«Interface»
TelefoneListener
void telefoneTocou(TelefoneEvent)
void telefoneAtendido(TelefoneEvent)
TelefoneAdapter
void telefoneTocou(TelefoneEvent)
void telefoneAtendido(TelefoneEvent)
Faz nada
Cria ^
Telefone
-tepefoneListeners:Vector
+tocaFone():void
+atendeFone():void
+addTelefoneListener(TelefoneListener):void
+removeTelefoneListener(TelefoneListener):void
-disparaTelefoneTocou():void
-disparaTelefoneAtendido():void
< Subscribe
Notify >
Pessoa
void telefoneTocou(TelefoneEvent)
void telefoneAtendido(TelefoneEvent)
SecretáriaEletrônica
void telefoneTocou(TelefoneEvent)
void telefoneAtendido(TelefoneEvent)

ETAPAS

1. Definir classes de categorias de eventos
2. Definir interfaces de Listener
3. Definir classes de adaptação (opcional)
4. Definir a classe observável Source
5. Definir objetos Listener

ETAPA 01

- **Definir classes de categorias de eventos**
  - Defina uma classe separada de evento para cada categoria principal de eventos que poderão ser gerados pelo Source
    - Nós teremos uma única categoria: "Telefone"
  - Faça com que cada classe de eventos estenda java.util.EventObject
    - **TelefoneEvent**

ETAPA 02

- **Definir interfaces de Listener**
  - Para cada categoria de eventos, defina uma interface que estenda **java.util.EventListener**
  - Esta interface deve conter um método para cada evento (da categoria) que vai gatilhar a propagação de informação do Source para os Listeners
  - Chame a interface como chamou a classe de eventos, com terminação Listener em vez de Event
    - TelefoneEvent → a interface de Listener é **TelefoneListener**

ETAPA 02

- **Definir interfaces de Listener**
  - Dê nomes aos métodos da interface para descrever a situação que causou o evento.
    - **telefoneTocou**
  - Cada método deve retornar **void** e aceitar um parâmetro, uma referência a uma instância da classe de eventos apropriada
    - **public void telefoneTocou(TelefoneEvent e);**

ETAPA 03

- **Definir classes de adaptação (opcional)**
  - É muito comum querer uma classe que implemente uma interface fazendo nada para a maioria dos métodos e fazendo algo útil apenas para alguns poucos métodos
  - Cria-se uma classe "adapter" que implemente todos os métodos e uma interface com métodos que nada fazem
  - Para cada interface de Listener que contenha mais do que um método, defina uma classe adapter que implemente a interface por inteiro com métodos que nada fazem
  - Dê um nome à classe com terminação Adapter em vez de Listener
    - TelefoneListener → **TelefoneAdapter**

ETAPA 04

- **Definir a classe observável Source**
  - Para cada categoria de eventos definir um par de métodos para adicionar/remover Listeners
    - **addTelefoneListener(TelefoneListener lis)**
    - **removeTelefoneListener(TelefoneListener lis)**
  - Para cada método em cada interface de Listener define um método privado de propagação de eventos
    - **disparaTelefoneTocou()**

ETAPA 04

- **Definir a classe observável Source**
  - Chamadas ao método de disparo de eventos devem ser adicionas em lugares apropriados da classe Source
    - **Nos lugares onde há mudança de estado**

ETAPA 05

- **Definir objetos Listener**
  - Para ser um Listener de uma certa categoria de eventos, basta implementar a interface de Listener da categoria de eventos
  - Isso é normalmente feito estendendo a classe Adapter, mas não é obrigatório

CÓDIGO
EVENTO: TelefoneEvent

TelefoneEvent.java

```java
1  package source;
2
3  import java.util.EventObject;
4
5  public class TelefoneEvent extends EventObject {
6
7      public TelefoneEvent(Telefone source) {
8          super(source);
9      }
10
11 }
```

- O source é passado como parâmetro e armazenado
- O listener recebe o evento e tem acesso ao source que o gerou
  - java.util.EventObject.getSource()
- Permite que um mesmo objeto seja Listener de vários objetos Source

CÓDIGO
INTERFARCE: TelefoneListener

TelefoneListener.java

```java
package listener;

import source.TelefoneEvent;

public interface TelefoneListener extends java.util.EventListener {

    void telefoneTocou(TelefoneEvent e);

    void telefoneAtendido(TelefoneEvent e);

}
```

Adapter (Opcional): TelefoneAdapter

TelefoneAdapter.java

```java
package adapter;

import listener.TelefoneListener;

public class TelefoneAdapter implements TelefoneListener {

    public void telefoneTocou(TelefoneEvent e) {}

    public void telefoneAtendido(TelefoneEvent e) {}
}
```

CÓDIGO
SOURCE: Telefone

Telefone.java

```
1  package source;
2
3  import java.util.ArrayList;..
6
7  public class Telefone {
8
9      private List<TelefoneListener> telefoneListeners = new ArrayList<TelefoneListener>();
10
11     // método de suporte para testar a solução
12     public void tocaFone() {
13         disparaTelefoneTocou();
14     }
15
16     // método de suporte para testar a solução
17     public void atendeFone() {
18         disparaTelefoneAtendido();
19     }
20
21     public synchronized void addTelefoneListener(TelefoneListener l) {
22         if (!telefoneListeners.contains(l)) {
23             telefoneListeners.add(l);
24         }
25     }
26
27     public synchronized void removeTelefoneListener(TelefoneListener l) {
28         telefoneListeners.remove(l);
29     }
```

CÓDIGO
SOURCE: Telefone

```
31      private void disparaTelefoneTocou() {
32          List<TelefoneListener> tl;
33          synchronized (this) {
34              // Clonar para evitar problemas de sincronização
35              // durante a propagação
36              tl = (List) (((ArrayList) telefoneListeners).clone());
37          }
38          TelefoneEvent evento = new TelefoneEvent(this);
39          for (TelefoneListener t : tl) {
40              t.telefoneTocou(evento);
41          }
42      }
43
44      // disparaTelefoneAtendido() é semelhante a disparaTelefoneTocou()
45      private void disparaTelefoneAtendido() {
46          List<TelefoneListener> tl;
47          synchronized (this) {
48              tl = (List) (((ArrayList) telefoneListeners).clone());
49          }
50          TelefoneEvent evento = new TelefoneEvent(this);
51          for (TelefoneListener t : tl) {
52              t.telefoneAtendido(evento);
53          }
54      }
55
56  }
```

CÓDIGO
LISTENER: SecretariaEletronica

SecretariaEletronica.java

```java
package listener;

import source.TelefoneEvent;

public class SecretariaEletronica implements TelefoneListener {

    public void telefoneTocou(TelefoneEvent e) {
        System.out.println("Secretaria escuta o telefone tocando.");
    }

    public void telefoneAtendido(TelefoneEvent e) {
        System.out.println("Secretaria sabe que o telefone foi atendido.");
    }
}
```

CÓDIGO
LISTENER: Pessoa

Pessoa.java

```java
package listener;

import source.Telefone;
import source.TelefoneEvent;

public class Pessoa implements TelefoneListener {

    public void telefoneTocou(TelefoneEvent e) {
        System.out.println("Eu atendo!!!");
        Telefone tel = (Telefone) e.getSource();
        tel.atendeFone();

    }

    public void telefoneAtendido(TelefoneEvent e) {
        System.out.println("Pessoa sabe que o telefone foi atendido.");
    }
}
```

CÓDIGO
MAIN: Aplicação

ExemploFone.java

```java
package main;

import listener.Pessoa;...

public class ExemploFone {

    public static void main(String[] args) {

        Telefone fone = new Telefone();
        Pessoa fulano = new Pessoa();
        SecretariaEletronica se = new SecretariaEletronica();

        fone.addTelefoneListener(fulano);
        fone.addTelefoneListener(se);

        fone.tocaFone();
    }

}
```

Pro @ Jav Dec Sea Con Cov

```
<terminated> ExemploFone [Java Application] /Library/Java/JavaVirtual
Eu atendo!!!
Pessoa sabe que o telefone foi atendido.
Secretaria sabe que o telefone foi atendido.
Secretaria escuta o telefone tocando.
```

QUANDO USAR?

- Quando uma abstração tem dois aspectos dependentes. Encapsular tais aspectos em objetos separados permite que variem e sejam reusados separadamente
- Quando uma mudança a um objeto requer mudanças a outros e você não sabe quantos outros objetos devem mudar
- Quando um objeto deve ser capaz de avisar outros sem fazer suposições sobre quem são os objetos (sem criar um acoplamento forte entre os objetos)

CONSEQUÊNCIAS DO USO

- Permite que se variem objetos Source e Listeners independentemente
  - Pode-se reusar objetos Source sem reusar seus Listeners e vice-versa
  - Pode-se adicionar Listeners sem modificar o Source ou os outros Listeners
- O acoplamento entre Source e Listeners é mínimo
  - Basta que os Listeners implementem uma interface simples
- Suporte para comunicação em broadcast
  - O Source faz broadcast do aviso. Os Listeners podem fazer o que quiserem com o aviso, incluindo ignorá-lo
- Do lado negativo: o custo de uma mudança ao estado de um Source pode ser grande se houver muitos Listeners

Dúvida?